

Московский государственный технический
университет им. Н. Э. Баумана

Факультет “Информатика и системы управления”
Кафедра ИУ5 “Системы обработки информации и управления”

Курс “Парадигмы и конструкции языков программирования”
Отчёт по лабораторной работе №1

Выполнил:
Студент группы ИУ5-31Б
Луценко Александра

Проверил:
Гапанюк Ю.Е

main.py

import sys

import math

def isFloat(value):

if value is None:

return False

valueStr = str(value).strip()

if not valueStr:

return False

if valueStr[0] in '+-':

valueStr = valueStr[1:]

hasDot = False

hasDigits = False

for char in valueStr:

if char == '.':

if hasDot:

return False

hasDot = True

elif char.isdigit():

hasDigits = True

else:

return False

return hasDigits

def stringToFloat(value):

if not isFloat(value):

return None

valueStr = str(value).strip()

sign = 1

if valueStr and valueStr[0] == '-':

sign = -1

valueStr = valueStr[1:]

elif valueStr and valueStr[0] == '+':

valueStr = valueStr[1:]

parts = valueStr.split('.')

if len(parts) == 1:

integerPart = parts[0]

return sign * convertInteger(integerPart)

elif len(parts) == 2:

```

integerPart = parts[0]
fractionalPart = parts[1]
integerValue = convertInteger(integerPart) if integerPart else 0
fractionalValue = convertFractional(fractionalPart) if fractionalPart else 0
return sign * (integerValue + fractionalValue)
else:
    return None

def convertInteger(integerStr):
    result = 0
    for char in integerStr:
        if char.isdigit():
            result = result * 10 + (ord(char) - ord('0'))
    return result

def convertFractional(fractionalStr):
    result = 0
    for char in fractionalStr:
        if char.isdigit():
            result = result * 10 + (ord(char) - ord('0'))
    return result / (10 ** len(fractionalStr))

def getValidCoefficient(prompt, value=None):
    while True:
        if value is not None:
            coefficient = stringToFloat(value)
            if coefficient is not None:
                return coefficient
            else:
                print(f"Некорректное значение коэффициента. {prompt}")
                value = None
                continue
        userInput = input(prompt)
        coefficient = stringToFloat(userInput)

        if coefficient is not None:
            return coefficient
        else:
            print("Ошибка! Введите действительное число.")

def solveBiquadratic():
    print("\n" + "Решение биквадратного уравнения  $A \cdot x^4 + B \cdot x^2 + C = 0$ ")
    print("=" * 50)
    args = sys.argv[1:]
    a_value = args[0] if len(args) > 0 else None

```

```

A = getValidCoefficient("Введите коэффициент A: ", a_value)
b_value = args[1] if len(args) > 1 else None
B = getValidCoefficient("Введите коэффициент B: ", b_value)
c_value = args[2] if len(args) > 2 else None
C = getValidCoefficient("Введите коэффициент C: ", c_value)

print(f"\nУравнение: {A}*x^4 + {B}*x^2 + {C} = 0")

if A == 0:
    print("Ошибка! Коэффициент A не может быть равен 0 для биквадратного уравнения.")
    return

D = B**2 - 4*A*C
print(f"Дискриминант D = {D}")

if D < 0:
    print("Уравнение не имеет действительных корней.")
    return

sqrt_D = math.sqrt(D)
t1 = (-B + sqrt_D) / (2*A)
t2 = (-B - sqrt_D) / (2*A)

print(f"Корни относительно t=x^2: t1 = {round(t1, 6)}, t2 = {round(t2, 6)}")

realRoots = []
if t1 >= 0:
    root1 = math.sqrt(t1)
    root2 = -math.sqrt(t1)
    realRoots.extend([root1, root2])

if t2 >= 0 and abs(t2 - t1) > 1e-10:
    root3 = math.sqrt(t2)
    root4 = -math.sqrt(t2)
    realRoots.extend([root3, root4])

uniqueRoots = []
for root in realRoots:
    isDuplicate = False
    for existingRoot in uniqueRoots:
        if abs(root - existingRoot) < 1e-10:
            isDuplicate = True
            break
    if not isDuplicate:

```

```

        uniqueRoots.append(root)
uniqueRoots.sort()

if not uniqueRoots:
    print("Уравнение не имеет действительных корней.")
else:
    print(f"Действительные корни уравнения: {[round(root, 6) for root in uniqueRoots]}")
    print(f"Количество действительных корней: {len(uniqueRoots)}")
    for i, root in enumerate(uniqueRoots, 1):
        print(f"Корень {i}: x = {[round(root, 6) for root in uniqueRoots]}")

def main():
    solveBiquadratic()
if __name__ == "__main__":
    main()

```

Решение биквадратного уравнения $A \cdot x^4 + B \cdot x^2 + C = 0$

=====

Введите коэффициент A: 2

Введите коэффициент B: 8

Введите коэффициент C: 2

Уравнение: $2 \cdot x^4 + 8 \cdot x^2 + 2 = 0$

Дискриминант D = 48

Корни относительно $t=x^2$: $t_1 = -0.267949$, $t_2 = -3.732051$

Уравнение не имеет действительных корней.

Решение биквадратного уравнения $A \cdot x^4 + B \cdot x^2 + C = 0$

=====

Введите коэффициент A: 4

Введите коэффициент B: 3

Введите коэффициент C: -3

Уравнение: $4 \cdot x^4 + 3 \cdot x^2 + -3 = 0$

Дискриминант D = 57

Корни относительно $t=x^2$: $t_1 = 0.568729$, $t_2 = -1.318729$

Действительные корни уравнения: $[-0.754141, 0.754141]$

Количество действительных корней: 2

Корень 1: x = $[-0.754141, 0.754141]$

Корень 2: x = $[-0.754141, 0.754141]$

Решение биквадратного уравнения $A \cdot x^4 + B \cdot x^2 + C = 0$

=====

Введите коэффициент A: 3

Введите коэффициент B: -5

Введите коэффициент C: 1

Уравнение: $3 \cdot x^4 + -5 \cdot x^2 + 1 = 0$

Дискриминант D = 13

Корни относительно $t=x^2$: $t_1 = 1.434259$, $t_2 = 0.232408$

Действительные корни уравнения: $[-1.197605, -0.482087, 0.482087, 1.197605]$

Количество действительных корней: 4

Корень 1: $x = [-1.197605, -0.482087, 0.482087, 1.197605]$

Корень 2: $x = [-1.197605, -0.482087, 0.482087, 1.197605]$

Корень 3: $x = [-1.197605, -0.482087, 0.482087, 1.197605]$

Корень 4: $x = [-1.197605, -0.482087, 0.482087, 1.197605]$